

Vanier College
Faculty of Science and Technology – Computer Science Department

Assignment 1

Build a simple Campus Equipment API

Individual work | 12.5% | Due Friday, October 9, 2026 at 11.29 AM

Target: about 3 hours including lab time, using your own working lab code.

This assignment must be completed individually. Write and test your own solution. You may discuss concepts, but do not share code, develop the submission together, or submit team code. You may reuse the supplied starter and your own acknowledged lab code.

Build a PHP API that lists, displays and adds equipment stored in MariaDB. Use the supplied single table and eight example items. The three tasks below retain the simplified v2 scope.

1 Build three endpoints

Use `/api/v1` as the base path. Each item has `id`, `name` and `category`. The database generates the ID. The name must be a string of 1–80 characters after trimming; the category must be the string `camera`, `audio` or `accessory`. Use prepared PDO statements for user-supplied values.

Request	Expected behaviour
GET <code>/items</code>	200 with the collection described in Task 2.
GET <code>/items/{id}</code>	200 with one item, or 404 if it does not exist.
POST <code>/items</code>	Accept name and category as JSON. Store the item; return 201 with the full item and a Location header pointing to its URI.

Example request

```
POST /api/v1/items
Content-Type: application/json

{"name":"Portable projector","category":"accessory"}
```

Return JSON using `Content-Type: application/json`. Reject malformed JSON, missing fields, incorrect types, a blank/overlong name or an invalid category with 400 and a simple message such as `{"error":"Name is required"}`. Never insert a row when validation fails. Use the same error shape for 404.

2 Add a small collection query

GET `/items` accepts optional category and page parameters. Apply an exact category filter when supplied; always order by ID ascending. Return two items per page, or fewer at the end. The page defaults to 1 and must be an integer from 1 to 100. An invalid category or page returns 400. A page with no results returns 200 with an empty data array.

Example response with the supplied data

```
GET /api/v1/items?category=camera&page=2

{"data":[{"id":7,"name":"Action camera","category":"camera"}],
"page":2,"pageSize":2,"total":3}
```

The matching IDs are 1, 2 and 7, so page 2 contains only item 7. Count all matching items **before pagination**. Apply the filter, ordering and page in SQL. No sorting control or pagination links are required.

3 Document, check and demonstrate

Complete the supplied OpenAPI file by adding response schemas and the Location header where indicated. Paths, parameters and schemas are already provided. Set your local server URL and check the file in an OpenAPI 3.1-compatible tool.

Five checks

Run these with curl, Postman or Insomnia after resetting the data. The requests are supplied. No screenshots or separate test report are required.

#	Request or action	Expected result
1	GET /items	200; IDs 1 and 2; total 8.
2	GET /items?category=camera&page=2	200; ID 7; total 3.
3	GET /items/999	404 with a JSON error message.
4	POST the supplied create.json, then GET the returned Location	201 + Location; retrieval returns the stored item.
5	POST {"category": "camera"}	400; name is missing; no row added.

AI use is permitted only for learning

AI may help you understand concepts or error messages. **Do not use AI to produce the submitted code or text.** You must write the implementation and explanations yourself. **Code or content fully or mostly generated by AI will be flagged for academic-integrity review.** An AI-use disclosure does not make such a submission acceptable. This restriction does not apply to the supplied starter or your own acknowledged lab code.

Include two short sections in your README

- **AI use statement (2–3 sentences):** name any AI tools used, their learning purpose, and what you wrote and verified yourself. If none were used, state: *No AI tools were used.*
- **How I did the work (4–6 sentences):** in your own words, without AI writing or rewriting this note, describe what you built first; refer to one file or function you wrote; explain one difficulty or design decision and how you handled it; and give one request you ran and the result you actually observed.

These are brief accounts of your actual work, not an extra report. No chat transcripts are required.

Submission and individual demonstration

Submit one ZIP on LEA with your PHP source, completed OpenAPI file, supplied database scripts and README. Include your name, local URL, setup steps, sources and the two short sections above. Include dependency files if needed; exclude vendor and real credentials.

Demonstrate your service **individually** in about 3–4 minutes: show a filtered page and a successful creation, then explain the relevant route and PDO query. Be ready to explain your development note and predict a small change. You must understand your submitted code to receive marks for that part.

Marking out of 100

Area	Marks	What counts
Working API	60	List, view and create (30); PDO persistence (15); validation and JSON responses (15).
Collection query	20	Category filter (8); fixed ID order and two-item pages (8); filtered total (4).
Documentation	10	OpenAPI starter (6); README setup and both short sections (3); clear prepared English (1).
Individual demo	10	Run the service and explain your own code, query and stored data.

Scope and dates. The course outline's late-work and academic-integrity procedures apply. See LEA for the exact upload time; the demonstration is in the October 8 lab.